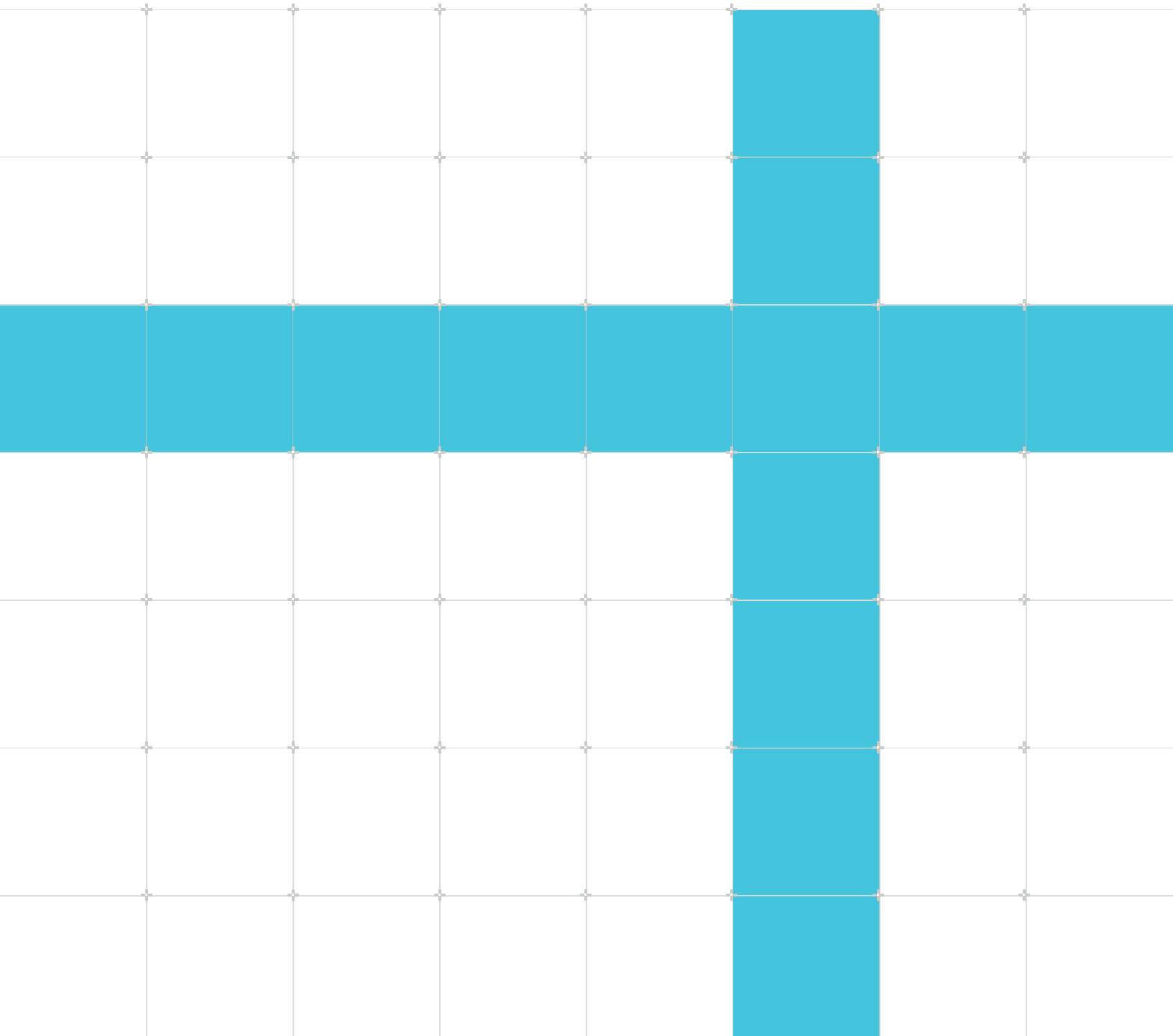
Learn the architecture - Introduction to AMBA® CHI C2C property negotiation

Version 1.0

Non-Confidential
Copyright © 2026 Arm Limited (or its affiliates).
All rights reserved.

Issue 01
112071_0100_01_en



Learn the architecture - Introduction to AMBA® CHI C2C property negotiation

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.

Release information

Document history

Issue	Date	Confidentiality	Change
0100-01	18 June 2026	Non-Confidential	Initial release

Proprietary Notice

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited ("Arm"). No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of this document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word "partner" in reference to Arm's customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm's trademark usage guidelines at https://www.arm.com/company/policies/trademarks. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

PRE-1121-V1.0

Confidentiality Status

This document is Non-Confidential. The right to use, copy and disclose this document may be subject to license restrictions in accordance with the terms of the agreement entered into by Arm and the party that Arm delivered this document to.

Unrestricted Access is an Arm internal classification.

Product Status

The information in this document is Final, that is for a developed product.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Feedback

Arm welcomes feedback on this product and its documentation. To provide feedback on the product, create a ticket on https://support.developer.arm.com

To provide feedback on the document, fill the following survey: https://developer.arm.com/documentation-feedback-survey.

Inclusive language commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used language that can be offensive. Arm strives to lead the industry and create change.

We believe that this document contains no offensive language. To report offensive language in this document, email terms@arm.com.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Contents

1. Overview of AMBA CHI C2C 7
1.1 Chiplets 7
1.2 Open chiplet ecosystem and the need for standards 8
1.3 AI and interconnects 8
1.4 AMBA CHI C2C 9

2. Interoperability and complexity scaling requirements 11
2.1 Complexity scaling 11
2.2 Interoperability 12

3. Property exchange and negotiation mechanism 13
3.1 The mechanism 13
3.2 Transmitter termination 14
3.3 Implementation 15

4. AMBA CHI C2C property categories 17
4.1 Transactions 17
4.2 Field width 17
4.3 Optimizations 18
4.4 Packetization 19
4.5 Cache stashing 19
4.6 Informational and debug 19
4.7 Future expansion 19
4.8 Additional properties 20

5. AMBA CHI C2C chiplet use cases 21
5.1 Accelerator attach 21
5.2 Advanced heterogeneous compute 22
5.3 Memory expansion 23

6. Conclusion 24

7. Related information 25

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

A. Properties and transactions.....26

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

1. Overview of AMBA CHI C2C

Building systems using chiplets, rather than building monolithic chips as a single block, lets designers mix and match best-in-class components. This lowers development costs by enabling composability, reuse, and performance scaling.

However, a chiplet ecosystem creating these systems requires open standards such as AMBA CHI C2C to support complexity scaling and interoperability.

Complexity scaling lets you connect chiplets with different features and capabilities. Examples range from simple read and write peripherals, to I/O or non-coherent device attach, to fully coherent advanced heterogeneous systems.

Interoperability ensures forward and backward compatibility across product generations, and between chiplets that might have been designed independently.

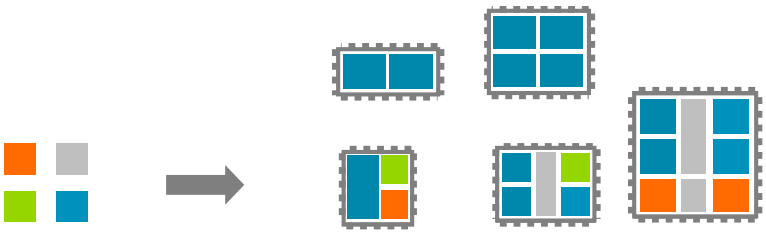
This document explains the property negotiation mechanism that AMBA CHI C2C provides to meet these requirements. The property negotiation mechanism uses a straightforward negotiation approach that you can easily implement in hardware. Combined with AMBA's ecosystem, dependability, and trust, property negotiation offers a reliable path for composable chiplets in the emerging open ecosystem.

1.1 Chiplets

Chiplets enable performance scaling, composability, reuse, and cost efficiency.

Rising design and manufacturing costs can limit innovation. Chiplets reduce SoC design costs by enabling reuse of a small set of general-purpose chiplets in a wide array of highly optimized and high-performance SoCs.

Figure 1-1: A small set of common chiplets can be combined in multiple topologies for different use cases



Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

1.2 Open chiplet ecosystem and the need for standards

A chiplet ecosystem requires open standards to enable design reuse and low-friction integration. Standards must be freely available, independent of architecture or platform, and widely adopted. Standards also need to be flexible and scalable so that they are applicable for a variety of designs.

More fundamentally, open standards are needed for interoperability between:

- Chiplets that have been designed independently
- Different chiplet types, with varying degrees of complexity
- Multiple generations of a design, to reduce design costs and maximize return on investment

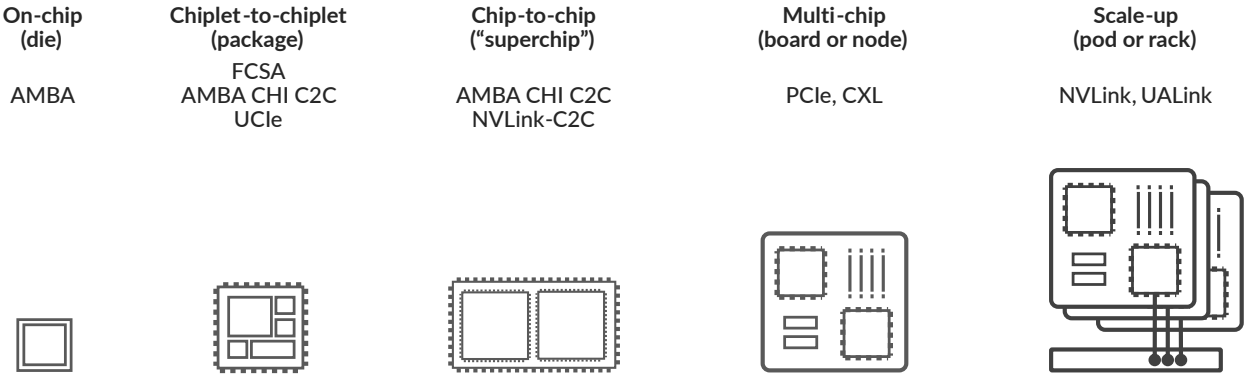
Standards must be complemented by a marketplace of compatible products, tools, and services that the industry can rely on. For chiplets, this means not only a marketplace of interoperable chiplets, but also a marketplace of interoperable IP products. For example, the processors, interconnects, controllers, and peripherals used to design chiplets.

1.3 AI and interconnects

AI workloads increase performance and system scalability requirements even further. Systems now scale across multiple integration levels. These include:

- Die: on-chip
- Package: chiplet-to-chiplet
- Node: superchips and chip-to-chip
- Rack: scale-up

Figure 1-2: Interconnect landscape and common standards today



Attaching an AI accelerator to a compute host is an important but challenging use case for interconnects across various integration levels. The rapid growth in AI model size is shifting the performance bottleneck to interconnect bandwidth.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

This trend makes the interconnect market landscape very dynamic and introduces even more challenging requirements for protocol standards:

- High performance: scalable interface bandwidth, high link efficiency, and low latency
- Flexibility: path to full coherency and unified memory
- Layered architecture: common protocol and latest PHY technology
- Open standards: interoperability and reduced risk

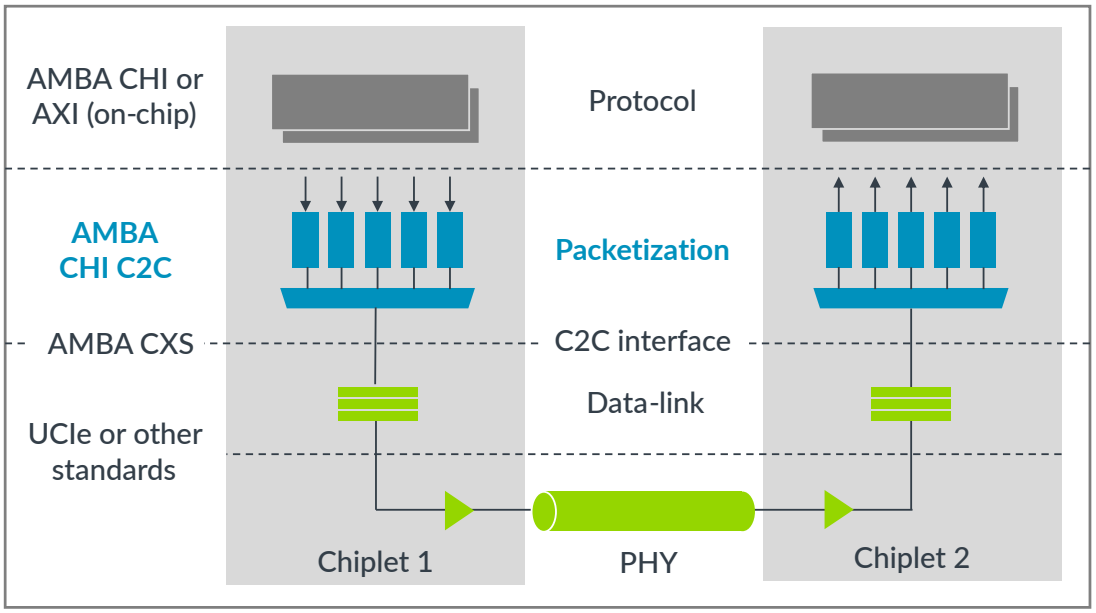
1.4 AMBA CHI C2C

AMBA is the open standard for SoC communications that underpins silicon IP and fabless design today. AMBA specifications like CHI and AXI have been at the heart of a widely used ecosystem and established marketplace with billions of devices shipped over almost 3 decades.

AMBA CHI C2C, also referred to as AMBA C2C or CHI C2C, extends the on-chip AMBA CHI protocol for connecting chip-to-chip and chiplet-to-chiplet (C2C). It is gaining momentum, particularly in chiplets and AI acceleration. AMBA CHI C2C complements the Foundation Chiplet System Architecture (FCSA), which defines how systems can be partitioned across chiplets.

AMBA CHI C2C extends AMBA for chip-to-chip applications. More specifically, it leverages the AMBA CHI protocol to provide the performance and scalability required for very large networks in C2C deployments. CHI is widely adopted, architecture neutral and has an extensive feature list. CHI is high-speed, credited, and packetized, which is ideal for C2C.

Figure 1-3: AMBA CHI C2C architecture overview



The AMBA CHI C2C specification focuses on packetization. AMBA CHI C2C uses shared channels to increase efficiency, and a simple packing mechanism to minimize latency and complexity.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Because it transports the same on-chip CHI protocol, it enables the seamless use of architecture features without protocol conversion.

AMBA CHI C2C follows a layered approach, so it can be used with transports (data-link and PHY) such as UCIe, other industry standards, or even bespoke solutions. For example, AMBA CHI C2C can be used over a UCIe Streaming interface, which provides link robustness through data-link CRC and retry.

The benefit is a single and unified C2C interface protocol for different use cases, maximizing interoperability and managing complexity, combined with the latest transport and PHY technologies, allowing re-use, best-in-class, and innovation at pace.

Existing designs using AMBA AXI on-chip can also benefit from AMBA CHI C2C by using an IO/non-coherent subset of the CHI protocol for their C2C links. The mapping between AXI transactions and this subset is straightforward, which allows for efficient on-chip bridging. You need only minor additional logic to handle differences in areas such as ID usage and ordering.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

2. Interoperability and complexity scaling requirements

AMBA CHI C2C enables complexity scaling and interoperability for C2C interfaces using a property exchange and negotiation mechanism.

The property exchange and negotiation mechanism supports use cases with different levels of complexity and requirements. Use cases range from simple devices that require read and write transactions to advanced heterogeneous systems that require high performance, full coherency, and advanced architecture features.

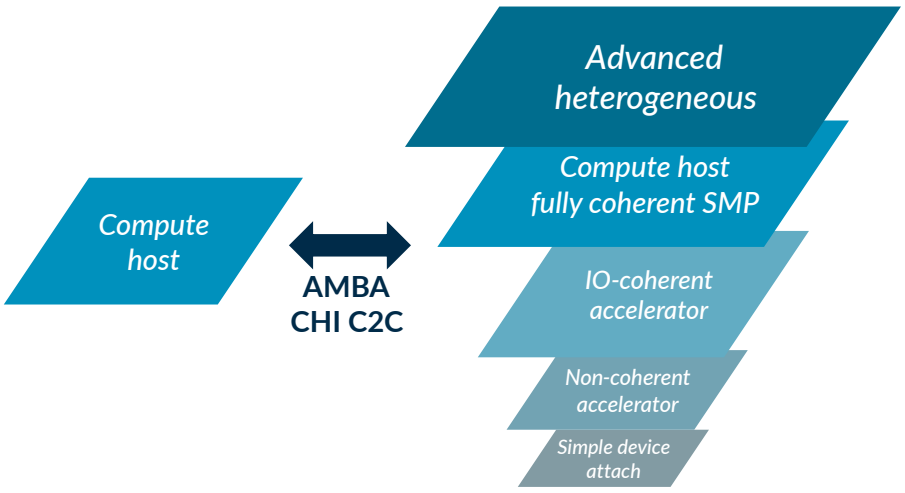
The property exchange and negotiation approach allows the use of a single unified interface protocol, maximizing interoperability between components and providing forward and backward compatibility.

2.1 Complexity scaling

Defining an appropriate interface complexity is challenging. High-end applications require many features and optimized transaction flows to maximize performance. This increases interface complexity. Simpler interfaces that support only read and write transactions do not require this level of complexity.

The property exchange and negotiation mechanism automatically configures two connected interfaces so that their complexity matches the use case.

Figure 2-1: Connecting one chiplet to various other types of chiplets with varying degrees of complexity through a single and unified interface protocol



For example, a simple peripheral expansion chiplet may not need full coherency. With all features disabled (and excluding link and protocol credit messages) there are only a few simple read and write opcodes that need to be supported:

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

- ReadNoSnp
- ReadOnce*
- WriteNoSnpFull/Ptl/Zero
- WriteUniqueFull/Ptl/Zero

Simplifying design complexity reduces design and verification effort, area, and power. A unified verification solution and methodology can be used for different use cases, further reducing costs and accelerating time-to-market. Reducing design complexity also makes it possible to lower power consumption by gating or powering off logic blocks associated with unused features.

Note

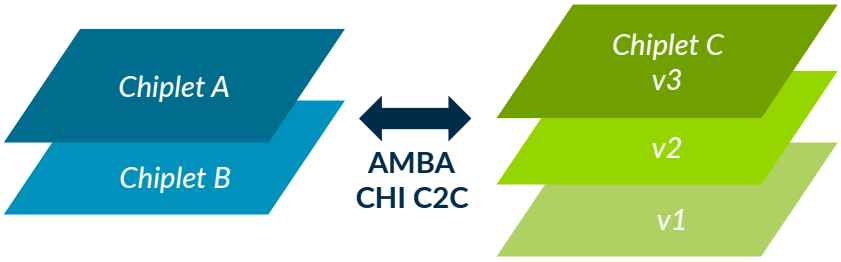
ReadOnce* represents collectively the ReadOnce, ReadOnceCleanInvalid, and ReadOnceMakeInvalid transactions.

2.2 Interoperability

Interoperability is critical when you connect chips or chiplets. These chiplets may:

- Be designed by different vendors
- Have different capabilities
- Evolve independently over time

Figure 2-2: Interoperability between different chiplet types and generations



The property negotiation process provides forward and backward compatibility as follows:

- A newer design must support older designs by downgrading unsupported features. For example, it must work with designs that do not recognize new features.
- Designs can rely on this behavior to remain compatible with future designs.

For each component, the property negotiation process scales design complexity to match its intended use. For example, a peripheral expansion chiplet might not require full coherency.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

3. Property exchange and negotiation mechanism

A property is a parameter used by an interface to declare its capabilities. Each interface across a C2C link advertises its supported features through properties. These properties are exchanged and negotiated between the two sides to determine the effective behavior of the link. The exchange and negotiation mechanism is described below.

3.1 The mechanism

The property exchange and negotiation mechanism uses 4 interface register sets:

- Supported
- Advertised
- Informed
- Negotiated

Each register set represents one step in the negotiation process. Table 3-1 shows the property register sets, and Figure 3-1 shows the negotiation mechanism.

The steps in the property exchange and negotiation process are as follows:

Supported

The Supported register set defines the hardware capabilities of an interface out of reset.

Each optional feature has an associated bit or bit field in the Supported register set. Hardware sets these bits to indicate supported and unsupported features.

Advertised

After reset, the Advertised register set matches the Supported register set. Firmware can then reduce the advertised feature set if required.

In many systems no downgrade occurs, and the Advertised registers remain identical to the Supported registers.

Some systems might choose to reduce the feature set through firmware during early boot. Firmware can downgrade features that the system does not require. This can improve interface efficiency. An example of downgrading might be to remove the need to send particular transactions, such as DVM messages, or downgrading by allowing a shorter message to be used, for example if stashing is not utilized. Another example is a multi-modal device that operates as either a host or an accelerator.

When the interface is initialized, the contents of the Advertised registers are sent using a dedicated MiscU.Properties message. This step is optional. Firmware can skip it when the connected device is known and the interface does not require message exchange. Even in

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

these situations, property exchange provides useful debug information during early system bring-up.

Informed

When the interface receives a MiscU.Properties message from the remote side, hardware records the values in the Informed register set. The Informed register set tells local firmware which capabilities the remote interface advertises.

Negotiated

Hardware combines the Informed register set with the local Advertised register set to create the Negotiated register set. The negotiation result determines which features the interface uses.

The actual negotiation mechanism is specific to each property. For example:

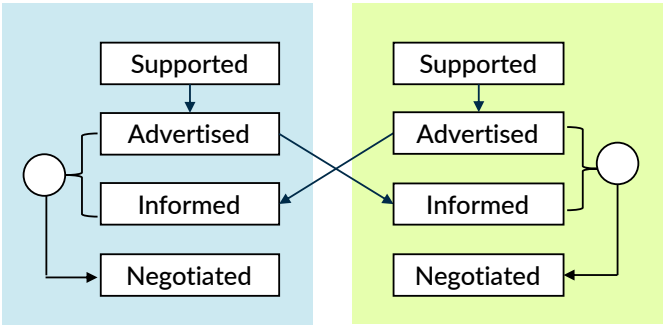
- For true or false options, the negotiation only results in the feature being supported if both sides of the interface indicate that they support it.
- Some properties, such as the maximum size of multi-request transaction, have numeric values associated with them and the result of the negotiation is the smaller of the two values.
- Some properties use specific negotiation rules that the AMBA CHI C2C specification defines.

After the negotiation process is complete, software can optionally check the results and report if there is a mismatch in expectations.

Table 3-1: Property register sets

Register set	Description
Supported	Defines hardware capabilities
Advertised	Defines locally advertised capabilities
Informed	Stores capabilities received from the remote interface
Negotiated	Stores the negotiated result

Figure 3-1: Property exchange and negotiation mechanism



3.2 Transmitter termination

AMBA CHI C2C uses a “Transmitter Termination” philosophy. This means that the Transmitter must prevent unsupported features from crossing the interface. When the Transmitter does not

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

support a feature, then no specific action is required because the Transmitter then does not use the feature. When the Receiver does not support a feature, the Transmitter must modify or terminate the related transaction. The AMBA CHI C2C specification does not require any particular action, but the Transmitter could take these actions:

- Adapt the transaction to something that is supported. For example, if cache stashing is not supported then a WriteUniqueFullStash transaction can be converted to a WriteUniqueFull transaction.
- Terminate the transaction locally and give an error response, such as NDERR, to indicate that a failure has occurred. Providing an error response to the transaction can help with identifying the source of the erroneous request.
- Terminate the transaction locally and give a sideband indication of the error condition, such as an interrupt or recording the details in a fault register.
- Terminate the transaction without reporting an error. This approach is not recommended because tracing any subsequent problems related to this action might be difficult.

Although the exact method used for Transmitter Termination is not specified, there are two fundamental requirements:

- A feature that is not supported must not be sent across the C2C interface.
- Any modified or terminated transaction must result in a transaction flow that is compliant with the on-chip protocol.

3.3 Implementation

Hardware implements the property exchange and negotiation mechanism. Firmware can disable the mechanism when required. The mechanism requires little logic and has a low implementation cost. For example, the actual negotiation is performed through either an AND operation or a simple compare logic.

The following example shows a scenario where a device such as an AI accelerator is attached to a compute host. Two separate property negotiations will then occur:

- The first involves the Host.Property.Tx and the Device.Property.Rx, which determines whether a transaction with a particular feature is sent from the Host to the Device.
- The second is in the opposite direction, involving the Device.Property.Tx and the Host.Property.Rx, which determines whether a transaction with a particular feature is sent from the Device to the Host.

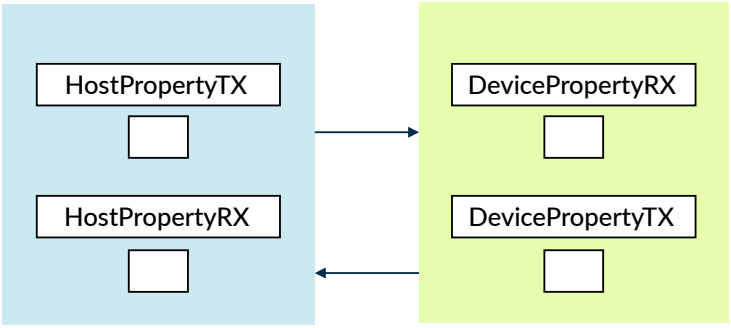
Property negotiation is directional, with independent negotiation for each transmit and receive path. This enables interfaces to use asymmetric capabilities.

For example, the Host may issue Atomic transactions, and the Device may support receiving them, enabling Atomics in the Host to Device direction. In contrast, if the Device does not generate Atomic transactions, the Device to Host direction will not use Atomics.

This asymmetry reflects real system behavior and avoids unnecessary restrictions on either interface.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Figure 3-2: Example of property negotiation for compute-accelerator attach



Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

4. AMBA CHI C2C property categories

The precise list of properties is defined in the AMBA CHI C2C specification. The following sections describe the main property categories.

4.1 Transactions

The following properties determine whether the interface supports a transaction group. Negotiation uses a logical AND operation between the Tx and Rx properties. The feature is enabled only when both values are true.

- Snoop_Transactions
- Atomic_Transactions
- Cache_Stash_Transactions
- Persist_Transactions
- Deferrable_Write
- RME_Support
- DVM_Support
- CMO_Transactions
- Physical_Storage_Transactions

The Snoop_Transactions property is particularly interesting because it can restrict CHI to either I/O-coherent or non-coherent transactions.

If an interface does not support receiving Snoop Transactions, it must not send transactions that allocate or deallocate fully coherent cache lines.

The appendix lists the transaction opcodes for each property.

4.2 Field width

The following properties determine whether the interface populates specific payload fields. Negotiation enables these properties only when both Tx and Rx values are True. Otherwise, negotiation sets the property to False.

- MPAM_Support
- PBHA_Support
- MEC_Support

If a feature is absent, the interface does not need to transport the associated field across the link.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

The following properties determine the width of a field, which determines whether a short or long message format is used.

Negotiation selects the smaller Tx or Rx value:

- RSVDC_REQ
- RSVDC_DAT

When packetization logic creates Request or Data messages, it can select either short or long message formats. Several factors determine the choice of message format, including:

- Presence of Memory system resource Partitioning And Monitoring (MPAM) fields
- Presence of Page-Based Hardware Attributes (PBHA) fields
- Presence of Memory Encryption Context (MEC) fields
- Required width of the RSVDC field

Packetization describes more packetization-related properties.

4.3 Optimizations

Properties can also negotiate optimized transaction flows for C2C communication:

- WritePush_Support
- DVMPush_Support

The AXI-like “write push” flow, for example, increases throughput efficiency by sending request and data information in the same C2C message. When the negotiation result is False, the Transmitter must use the default “write pull” flow.

The following properties define the maximum size of Multi-request transactions. The negotiation result is the lower of Tx and Rx values. If negotiation returns 0, the interface must not use that transaction flow.

- MultiReq_WritePull_Support
- MultiReq_WritePush_Support
- MultiReq_NonWrite_Support

CacheLineID_Accurate also affects Multi-request behavior. If any of the above properties permit the use of multi-request transactions, then this property is also True.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

4.4 Packetization

Some properties use more complex negotiation rules. The following list summarizes these rules. See the AMBA CHI C2C specification for full details.

- Container_Format - Negotiates the container format for the interface. The property can indicate which of Format X, Format Y, and Format Z are supported by both sides of the interface:
  - If only one format is supported by both sides, then that format is selected.
  - If more than one format is supported by both sides, the interface selects formats in this order: Format X, Format Z, then Format Y. The order corresponds to the number of bytes that each format reserves for the link layer.
- Deactivation_Support - Negotiation evaluates this property on a bitwise basis, rather than as a numeric value. This property does not directly affect hardware functionality. The property can be used to indicate to software controlling the de-activation sequence what level of interface quiescence is required.
- PortID - This passes the local PortID of an interface to the opposite side.

4.5 Cache stashing

Three additional properties control cache stashing:

- Cache_Stash_Transactions
- Cache_Stash_Requests
- Cache_Stash_Snoops

Each property enables a different set of stashing transactions.

4.6 Informational and debug

Informational properties support debug activities. The Protocol field currently only provides a value to indicate the C2C protocol but supports future protocol expansion. The Version field indicates the version of the specification that the component supports. Currently supported values are Issue A and Issue B. This property is intended for informational purposes only. Software should not use this property to disable features automatically that are not included in a particular version of the specification.

4.7 Future expansion

The Num_Properties field exists in the first Property exchange message and supports future expansion of the property exchange mechanism. This field enables future property exchange messages while maintaining compatibility with legacy designs. If a legacy design does not exchange additional property messages, it does not support any of the features that would be enabled by those additional property messages.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

4.8 Additional properties

The following properties provide additional functionality:

- MTE_Support - Determines whether memory tagging is supported and whether the Match operation can be used.
- DevAssign_Support - Determines the level of transaction checking that is performed when an untrusted device is connected to a host.
- Event_Support - Determines whether Event messages are supported using the MiscU.Event message.
- DataTarget_Support - Determines whether the permitted values for the DataTarget field in a Request message must be constrained to those defined in AMBA CHI Issue G (CHI-G), or if the expanded set defined in CHI-H can be used.

See the AMBA CHI C2C specification for more details.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

5. AMBA CHI C2C chiplet use cases

These examples show how negotiated properties tailor AMBA CHI C2C to common chiplet use cases, from accelerator attach to coherent compute and memory expansion.

5.1 Accelerator attach

Accelerator attach is a common chiplet use case. A compute host chiplet connects to a third-party accelerator chiplet that is typically I/O-coherent or non-coherent. The accelerator chiplet can have internal working memory, but the compute host and operating system (OS) cannot address that memory. In this use case, the system does not require many of the CHI protocol features or transactions, and can use properties to disable these features.

Figure 5-1: Example of a compute host attached to an I/O or non-coherent accelerator

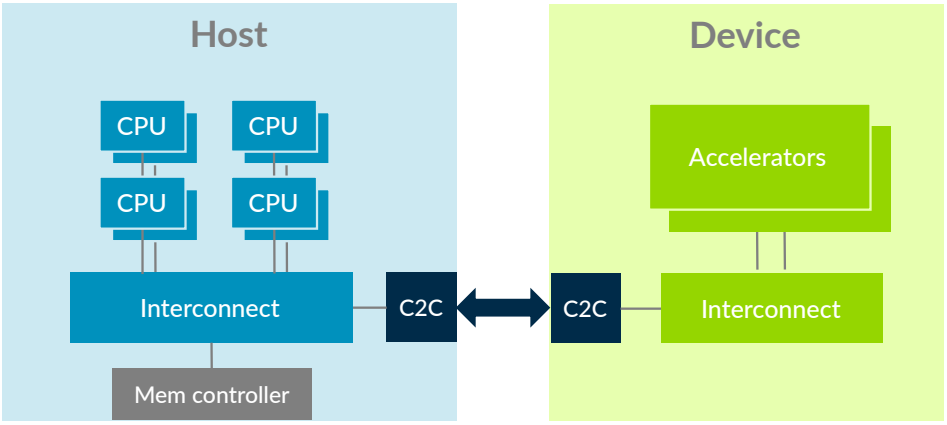


Table 5-1: Example properties for I/O-coherent or non-coherent accelerator attach

Property	Host → Device	Host ← Device
Snoop_Transactions	No	No
Atomic_Transactions	No	Optional
Cache_Stash_Transactions	No	Optional
Persist_Transactions	No	Optional
Deferrable_Write	No	Optional
RME_Support	No	Optional
DVM_Support	Yes	No
MTE_Support	No	Optional

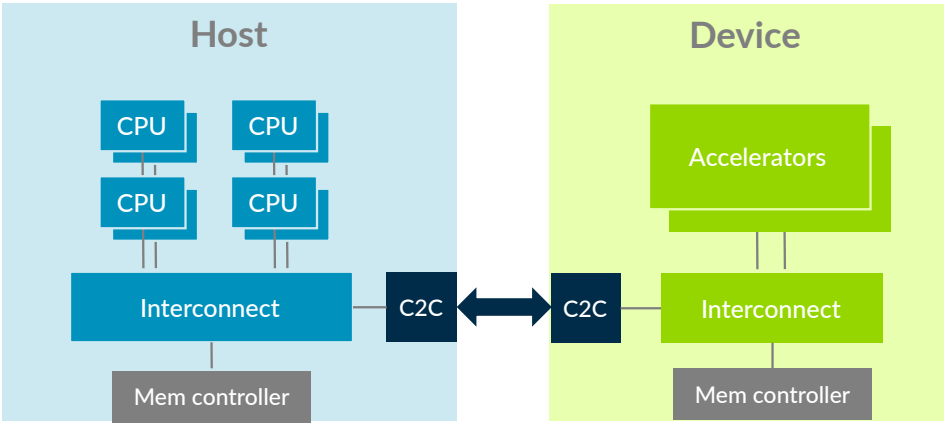
The Snoop_Transactions property disables snoop transactions and reduces CHI to an I/O-coherent or non-coherent protocol subset. This subset has lower complexity and is equivalent to AMBA AXI. The accelerator chiplet therefore only needs to implement the required features, which reduces area and cost.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

5.2 Advanced heterogeneous compute

Advanced heterogeneous compute is another important chiplet use case. Some vendors refer to this configuration as a super chip. In this configuration, a compute host connects to a third-party fully coherent accelerator that uses shared memory visible to the operating system.

Figure 5-2: Example of advanced heterogeneous compute with a fully coherent accelerator and shared memory



In this use case, compute and acceleration operate together. They share a unified memory space and require full coherency. The system frequently exchanges small data elements and requires synchronization between devices. The system also relies on advanced architectural and protocol features to achieve high performance. Properties are used to configure the precise set of required features.

Table 5-2: Example properties for an advanced heterogeneous compute system

Property	Host → Device	Host ← Device
Snoop_Transactions	Yes	Yes
Atomic_Transactions	Optional	Optional
Cache_Stash_Transactions	Optional	Optional
Persist_Transactions	Optional	Optional
Deferrable_Write	Yes	Yes
RME_Support	Optional	Optional
DVM_Support	Yes	Yes
MTE_Support	Optional	Optional

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

5.3 Memory expansion

Memory expansion is another important and emerging chiplet use case. This use case requires only a minimal set of protocol features and transactions. Properties configure the required features.

Figure 5-3: Example memory expansion configuration and associated properties

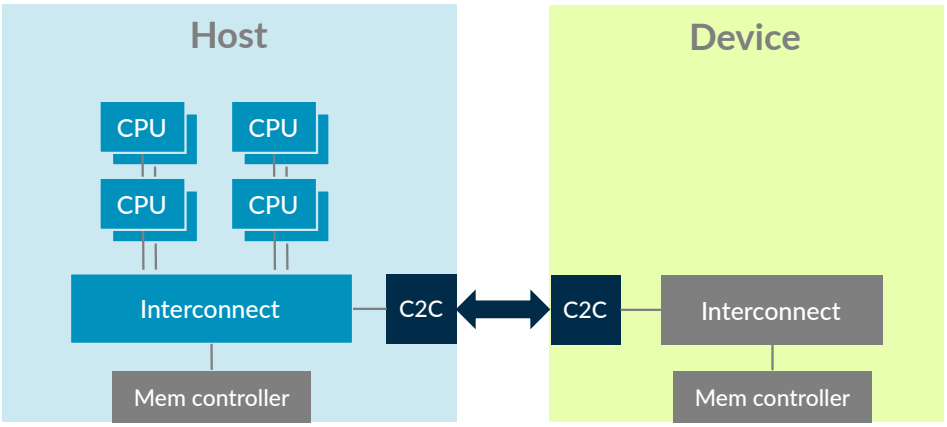


Table 5-3: Example of properties used for memory expansion

Property	Host → Device	Host ← Device
Snoop_Transactions	No	No
Atomic_Transactions	Optional	No
Cache_Stash_Transactions	No	No
Persist_Transactions	Optional	No
Deferrable_Write	Optional	No
RME_Support	Optional	No
DVM_Support	No	No
MTE_Support	Optional	No

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

6. Conclusion

AMBA CHI C2C property negotiation enables a single, unified interface protocol. AMBA CHI C2C supports a wide range of chiplet use cases with different complexity levels, from simple device and accelerator connections to advanced heterogeneous systems.

Property negotiation configures protocol subsets based on supported features. Properties ensure interoperability between chiplets with varying capabilities, vendors, and product generations.

This approach enables chiplet composability and performance scaling. It also provides forward and backward compatibility for scalable chiplet systems.

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

7. Related information

Here are some resources that are related to material in this guide:

- AMBA
- AMBA CHI C2C
- Foundation Chiplet System Architecture (FCSA)
- AMBA CHI C2C specification

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Appendix A Properties and transactions

This reference appendix lists the transaction opcodes associated with key AMBA CHI C2C properties, together with the corresponding conversion or termination behavior.

Property	Transaction opcode	Conversion or termination
Snoop_Transactions	ReadUnique	ReadNoSnp equivalent
	ReadPreferUnique	
	ReadClean	
	ReadShared	
	ReadNotSharedDirty	
	MakeReadUnique	
	WriteBackFull	WriteNoSnp equivalent
	WriteBackPtl	
	WriteCleanFull	
	WriteEvictFull	Terminate
	WriteEvictOrEvict	
	CleanUnique	
	MakeUnique	
	Evict	
	WriteBackFullCleanInv *	Combined WriteNoSnpFull equivalent
	WriteBackFullCleanInvPoPA *	
	WriteBackFullCleanInvStrg *	
	WriteBackFullCleanSh *	
	WriteBackFullCleanShPerSep *	
	WriteCleanFullCleanSh *	
	WriteCleanFullCleanShPerSep *	
Atomic_Transactions	AtomicCompare	Terminate
	AtomicLoad	
	AtomicStore	
	AtomicSwap	
Deferrable_Write	WriteNoSnpDef	Terminate
Cache_Stash_Transactions	StashOnceSepShared	Terminate
	StashOnceSepUnique	
	StashOnceShared	
	StashOnceUnique	
	WriteUniqueFullStash	WriteUniqueFull
	WriteUniquePtlStash	
CMO_Transactions	CleanInvalid	Terminate
	CleanShared	
	MakeInvalid	

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential

Property	Transaction opcode	Conversion or termination
	WriteNoSnpFullCleanInv	WriteNoSnpFull/Ptl
	WriteNoSnpFullCleanSh	
	WriteNoSnpPtlCleanInv	
	WriteNoSnpPtlCleanSh	
	WriteUniqueFullCleanSh	
	WriteUniquePtlCleanSh	
	WriteBackFullCleanInv *	
	WriteBackFullCleanSh *	
	WriteCleanFullCleanSh *	
Persist_Transactions	CleanSharedPersist	Terminate
	CleanSharedPersistSep	
	WriteNoSnpFullCleanShPerSep	WriteNoSnpFull
	WriteNoSnpPtlCleanShPerSep	WriteNoSnpPtl
	WriteUniqueFullCleanShPerSep	WriteUniqueFull
	WriteUniquePtlCleanShPerSep	WriteUniquePtl
	WriteBackFullCleanShPerSep *	WriteBackFull
	WriteCleanFullCleanShPerSep *	WriteCleanFull
RME_Support	CleanInvalidPoPA	Terminate
	WriteNoSnpFullCleanInvPoPA	WriteNoSnpFull
	WriteNoSnpPtlCleanInvPoPA	WriteNoSnpPtl
	WriteBackFullCleanInvPoPA *	WriteBackFull
Physical_Storage_Transactions	CleanInvalidStorage	Terminate
	WriteNoSnpFullCleanInvStrg	WriteNoSnpFull
	WriteUniqueFullCleanInvStrg	WriteUniqueFull
	WriteBackFullCleanInvStrg *	WriteBackFull
DVM_Support	DVMReq	Terminate
	DVMSync	

Note

- Entries shown with an asterisk appear multiple times in the table

Copyright © 2026 Arm Limited (or its affiliates). All rights reserved.
Non-Confidential